

영상처리

12 주차 보고서

컴퓨터융합학부

202102699 정민경

1. 과제 설명

함수이름	my_bilateral()
코드설명	bilateral filtering 수식으로 filtering 을 진행하는 함수이다. 이 때 parameter 정보를 보면 각각의 masksize(msize), 각각의 sigma, 관심패치의 x,y좌표, padding 유형('zero padding' 으로 진행), 그리고 마지막으로 "is_dog" 라는 인자가 들어온다. 이 인자는 bilateral filtering 을 할 때 Dog 항의 사용 유무를 결정하는 Boolean 값이다. 그래서 이 값이 true 이면, gaussian, (중앙값 Intensity - filter 각각의 위치의 Intensity) 에 dog 항의 수식까지 곱해 filtering 을 진행하면 되고, false 이면 gaussian, (중앙값 Intensity - filter 각각의 위치의 Intensity) 만으로 filtering 을 진행하면 된다. (이 값은 조건문으로 구현) Src의 크기만큼 이중 for문을 돌며 진행하고, 실습자료에 있는 수식을 바탕으로 구현했다. 먼저 gaussian 수식이다. 이 수식은 src[l,j] 즉, 우리가 src를 돌면서 관심있는 값에서부터 filter 각각의 위치가 얼마나 떨어진지를 계산해주는 수식이다. 위치 차이는 이미 위에 x, y 를 각각 mgrid 로 구해냈기 때문에 이 값을 (i-k)와 (j - l) 로 보고 구현해주었다. 두번째로는 intensity 의 차이를 사용하는 수식이다. 우리는 위에서 기존이미지인 src 에 msize의 절반만큼을 padding으로 채워주었다. 즉 src[l,j] 가 filter의 중간값이 된다. 그래서 이 수식을 구현할 때 우리가 바라보고있는 값에서 padding된 image 의 [l, j] 부터 [i+msize, j+msize] 만큼을 indexing 방법으로 돌며 수식을 구현해주었다. 마지막으로 dog 수식부분이다. 이 부분은 두번째 수식인 intensity 의 차이를 사용하는 수식과 동일한 방법으로 구현하고자 하였다. 마지막 수식은 dog 를 사용하는 수식이기 때문에 기존 src 와 padding 된 src 를 사용하는 것이 아닌 dog filtering 이 된 dog_img 와 padding 된 dog_img 로 사용해 수식을 구현하였다. Is_dog 값을 고려해 mask 를 구해준 뒤 밝기 차이를 두지 않기 위해 즉, mask 의 합을 1로 맞추기 위해 normalization을 진행하였다.
이미지	<pre> for i in range(h): print('\r%d / %d ...' %(i, h), end="") for j in range(w): # 실습자료에 있는 exp 수식으로 구현 # mask = ??? if (is_dog): gau_value = np.exp(-((x * x) + (y * y))) / (2 * sigma_s * sigma_s) intensity_value = np.exp(-(((src[i, j] - img_pad[i:i + msize, j:j + msize]) ** 2) / (2 * (sigma_r ** 2)))) dog_value = np.exp(-(((dog_img[i, j] - dog_pad[i:i + msize, j:j + msize]) ** 2) / (2 * (sigma_d ** 2)))) mask = gau_value * intensity_value * dog_value else: gau_value = np.exp(-((x * x) + (y * y))) / (2 * sigma_s * sigma_s) intensity_value = np.exp(-(((src[i, j] - img_pad[i:i + msize, j:j + msize]) ** 2) / (2 * (sigma_r ** 2)))) mask = gau_value * intensity_value # 정규화 => 합을 1로 만들기 위함. mask = (1 / np.sum(mask)) * mask </pre>

함수이름	main()
코드설명	main 함수에서는 filter size 와 첫번째, 두번째 수식에 사용되는 sigma 값을 찾아 주면 완성이 되는 코드이다. Filter size 와 sigma_s 값은 바로 아래 코드에서 my_get_Gaussian2D_mask() 를 호출 시 사용되는 size 와 sigma 값인 것으로 추측해 사용하였고, sigma_r 값 같은 경우 직접 하나씩 찍어보며 제일 비슷한 값으로 0.25를 사용하였다.
이미지	<pre> dst = my_bilateral(src_noise, 15, sigma_s=5, sigma_r=0.25, sigma_d=0.08, sigma_dog=3, pos_x=pos_x, pos_y=pos_y) dst = my_normalize(dst) # dog_dst = my_bilateral(src_noise, ??, sigma_s=??, sigma_r=??, sigma_d=0.08, sigma_dog=3, pos_x=pos_x, pos_y=pos_y, # pad_type='zero', is_dog=True) dog_dst = my_bilateral(src_noise, 15, sigma_s=5, sigma_r=0.25, sigma_d=0.08, sigma_dog=3, pos_x=pos_x, pos_y=pos_y, pad_type='zero', is_dog=True) dog_dst = my_normalize(dog_dst) </pre>

2. 결과 이미지

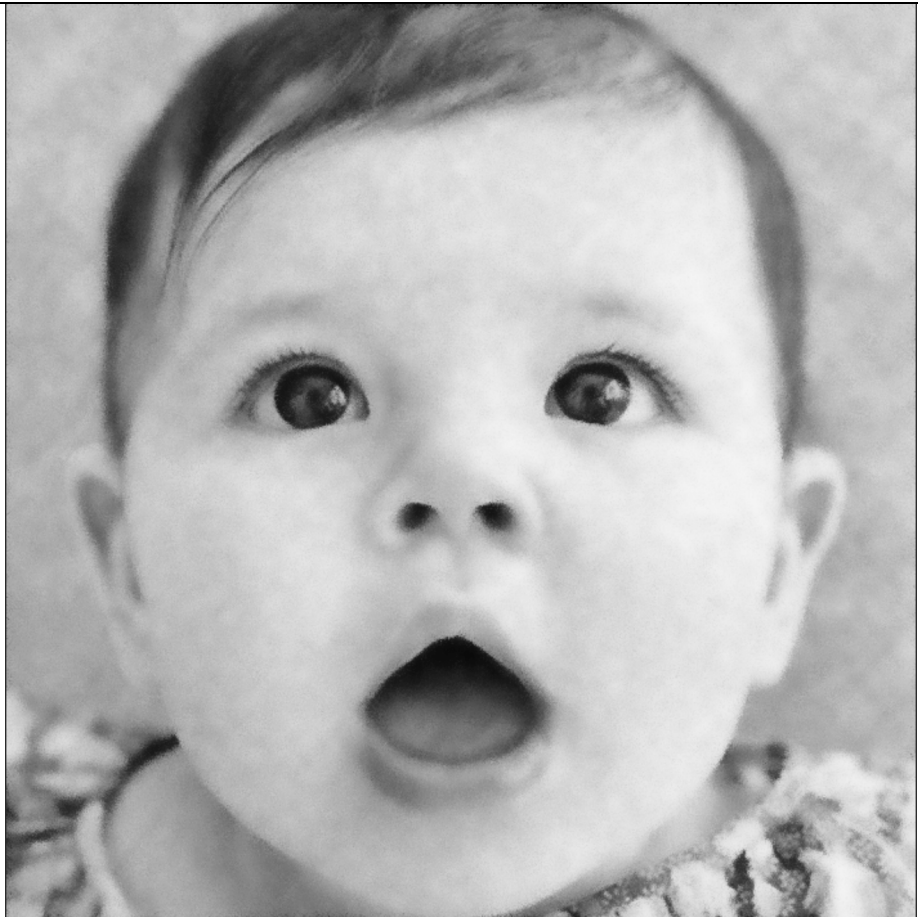
**Gaussian
Noise**



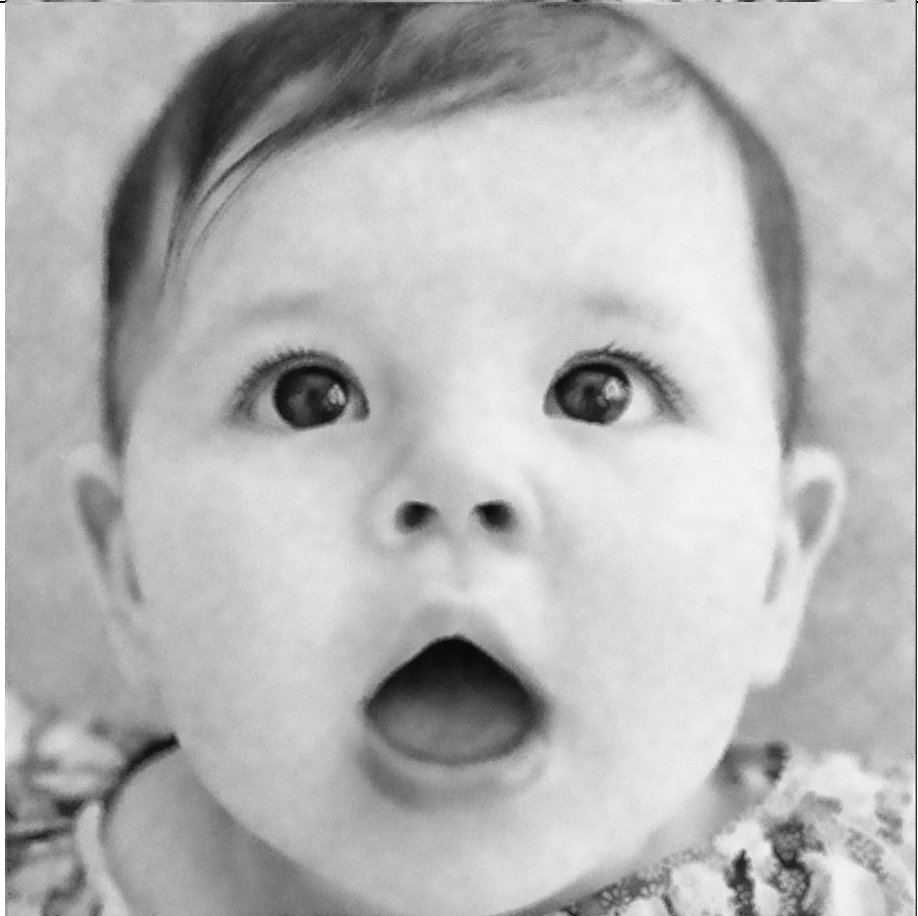
**Gaussian
Filtering**

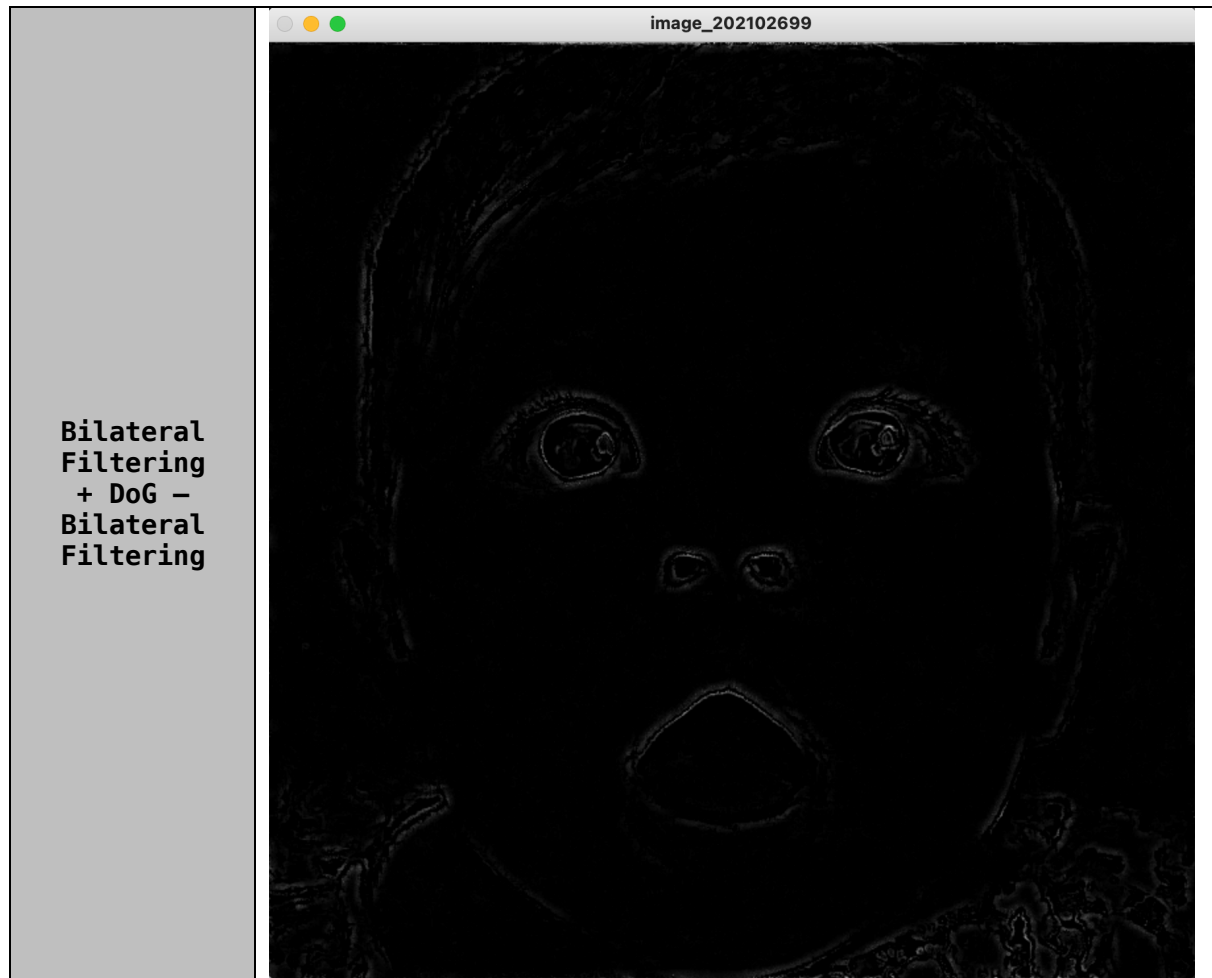


**Bilateral
Filtering**



**Bilateral
Filtering
+ DoG**





3. 같이 구현한 친구

202102695 장진영